MSA-DATB09-03 · NEURAL NETWORKS & DEEP LEARNING FOUNDATIONS

Session 7 / 8

Convolutional neural networks

Convolution and pooling by hand, from LeNet to ResNet, then transfer learning.

Guillaume Saint-Cirgue Albert School · MSc 1 October 2026

Guillaume Saint-Cirgue your instructor

Data Scientist and ML Engineer since 2016

2013 to 2016	Engineering degree, **ENSMM** (Besançon)
2016 to 2017	Data Scientist, **Rolls-Royce**
2017 to 2019	Lead ML Engineer, **GKN Aerospace**
2019	Launched **Machine Learnia** on YouTube: 200,000+ subscribers today
2020	Founded my company and my startup: **30+ AI projects** delivered since
2023	Head of Data, **Airbus**
Today	**ML-PRO** training program: 800 students

To go further

MY YOUTUBE CHANNEL

Machine Learnia

200,000+ subscribers. Machine learning and deep learning videos, including a neural network coded from scratch in NumPy.

youtube.com/@MachineLearnia

MY EBOOK

Apprendre le Machine Learning en une semaine

In English: Learn Machine Learning in One Week.

machinelearnia.com/apprendre-le-machine-learning-en-une-semaine

Both resources are in French. Click the cards in your copy of the course to open them.

Last sessions we flattened the images. What did we lose?

In sessions 5 and 6, a 28 × 28 image became a vector of 784 numbers, fed to dense layers. Three problems:

1. **Too many parameters.** Every neuron is connected to every pixel. On a real photo (224 × 224 × 3) this explodes.
2. **No neighbourhood.** Pixel 1 and pixel 2 are neighbours, pixel 1 and pixel 29 too (one row below): the dense layer does not know it. Shuffle the pixels the same way in every image and it learns exactly as well.
3. **No translation.** A boot detected in the centre must be learned again, with other weights, for every position.

What we want

- Look at **small neighbourhoods** (edges, textures are local).
- Use the **same detector everywhere** in the image.
- Combine local patterns into larger ones, layer after layer.

This is exactly what a convolutional layer does (LeCun et al., 1989 to 1998).

Exercise 1 · A dense layer on a photo

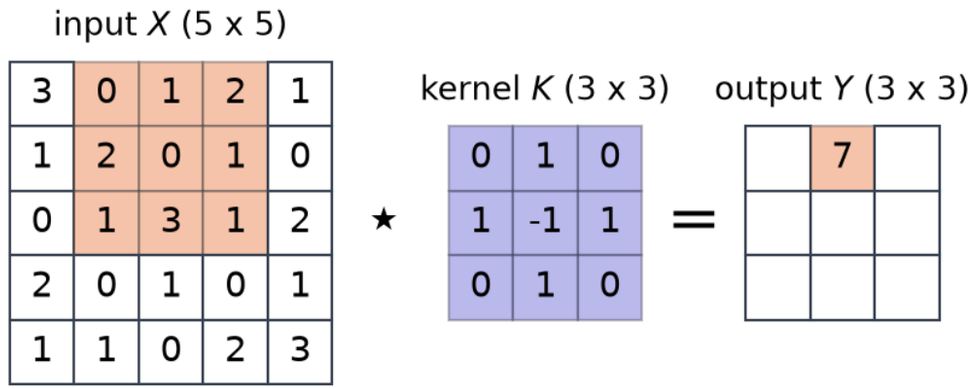
A colour photo of 224 × 224 pixels (3 channels) is flattened and fed to a dense layer of 1000 neurons.

Exercise 6 min pen and paper

1. How many inputs does the layer have?
2. How many parameters (weights and biases)?
3. How much memory to store them in float32 (4 bytes each)?
4. Compare: a convolution layer with 64 filters of size 3 × 3 on the same 3 channels has $64 \times (3 \times 3 \times 3 + 1)$ parameters. How many? Does it depend on the image size?

Your notes / answers (saved in this browser)

The convolution: a small filter slides over the image

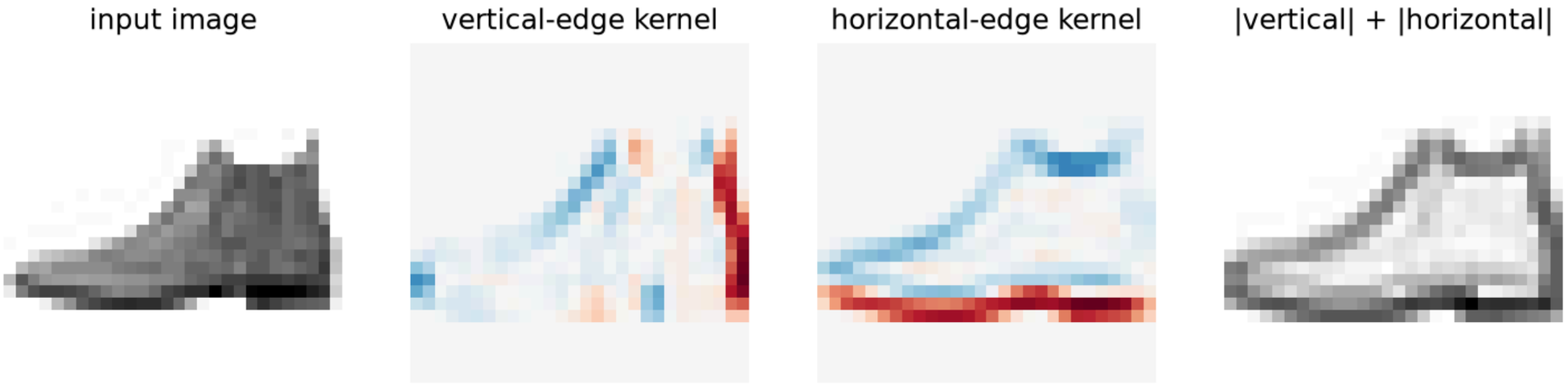


$$Y_{0,1} = \sum_{u,v} K_{u,v} X_{0+u,\,1+v} = 0\cdot 0 + 1\cdot 1 + 0\cdot 2 + 1\cdot 2 - 1\cdot 0 + 1\cdot 1 + 0\cdot 1 + 1\cdot 3 + 0\cdot 1 = 7$$

$$Y_{i,j} = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K_{u,v}\; X_{i+u,\,j+v} \;+\; b$$

- Place the $k \times k$ kernel on a patch, multiply element-wise, sum (plus a bias b), move by one pixel, repeat.
- Without padding, an $n \times n$ image gives an $(n-k+1) \times (n-k+1)$ output, the **feature map**.
- Strictly this is a cross-correlation (a true convolution flips the kernel); deep learning calls it convolution, and since K is learned the flip does not matter.

A filter is a pattern detector



Vertical-edge kernel $\begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}$ and its transpose on a FashionMNIST boot (red: positive, blue: negative).

- The output is large where the patch looks like the kernel: here, where the intensity changes from left to right.
- In a CNN the kernels are **not** designed by hand: they are weights, learned by backpropagation like any other.
- One kernel = 9 weights, used at every position: this is **weight sharing**.

Exercise 2 · A convolution by hand

$$X = \begin{pmatrix} 1 & 1 & 1 & 4 & 4 \\ 1 & 2 & 1 & 4 & 4 \\ 1 & 1 & 1 & 4 & 4 \\ 1 & 1 & 2 & 4 & 4 \\ 1 & 1 & 1 & 4 & 4 \end{pmatrix}$$

$$K = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}$$

Exercise 10 min pen and paper

1. What is the size of the output (no padding, stride 1)?
2. Show that, for this kernel, each output is (sum of the left column of the patch) minus (sum of its right column).
3. Compute the whole output Y.
4. Where is $|Y|$ large? What does this kernel detect? What would $-K$ give?

Your notes / answers (saved in this browser)

Padding and stride: the output size

- **Padding** p: add p rows and columns of zeros around the image, so that the borders are seen as often as the centre and the size can be kept.
- **Stride** s: move the kernel by s pixels instead of 1. Stride 2 halves the size.

Counting the positions: the padded image has $n + 2p$ pixels per row; the kernel fits in the first $n + 2p - k + 1$ positions, and we keep one position every s:

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$$

n	k	p	s	n_{out}
5	3	0	1	3
28	3	1	1	28 ("same")
28	2	0	2	14
224	7	3	2	112

"Same" padding for an odd kernel: $p = (k - 1)/2$, so 1 for 3 × 3, 3 for 7 × 7.

Channels: filters see all input channels

- A colour image has 3 channels (R, G, B); a hidden layer has many (one per filter of the previous layer).
- A filter has the shape $C_{\text{in}} \times k \times k$: it spans **all** input channels and produces **one** feature map.
- C_{out} filters give C_{out} feature maps: the output has C_{out} channels.

$$\#\text{params} = C_{\text{out}} \times (C_{\text{in}} \times k \times k + 1)$$

Independent of the image size n.

```python
conv = nn.Conv2d(in_channels=3, out_channels=16,
                 kernel_size=3, stride=1, padding=1)
print(conv.weight.shape)   # torch.Size([16, 3, 3, 3])
print(conv.bias.shape)     # torch.Size([16])

x = torch.randn(8, 3, 32, 32)   # 8 RGB images
print(conv(x).shape)       # torch.Size([8, 16, 32, 32])
pool = nn.MaxPool2d(2)
print(pool(conv(x)).shape) # torch.Size([8, 16, 16, 16])
```

PyTorch images are `(N, C, H, W)`: batch, channels, height, width.

Exercise 3 · Shapes and parameters along a network

Input: one grey image 1 × 28 × 28. Give the output shape (C, H, W) and the number of parameters after each layer.

Exercise 8 min pen and paper

1. `nn.Conv2d(1, 8, 3, padding=1)`
2. `nn.MaxPool2d(2)`
3. `nn.Conv2d(8, 16, 3)` (no padding)
4. `nn.MaxPool2d(2)`
5. `nn.Conv2d(16, 32, 3, stride=2, padding=1)`
6. `nn.Flatten()`: how many numbers reach the next dense layer?

Your notes / answers (saved in this browser)

Pooling: summarise each neighbourhood

- **Max pooling** 2 × 2, stride 2: keep the maximum of each 2 × 2 window. **Average pooling**: keep the mean.
- Halves the height and the width (4 times fewer values), so the next layers cost less.
- **No parameters**: nothing to learn.
- Some **translation invariance**: if a feature moves by one pixel inside its window, the maximum does not change.
- Typical block: conv, ReLU, conv, ReLU, pool. The number of channels usually doubles when the size halves.

$$\begin{pmatrix} 1 & 0 & 2 & 4 \\ 3 & 2 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 2 & 0 & 3 & 1 \end{pmatrix} \xrightarrow{\max 2\times 2} \begin{pmatrix} 3 & 4 \\ 2 & 3 \end{pmatrix}$$

Each output value is the largest of the 4 values of its window.

Exercise 4 · Pooling by hand

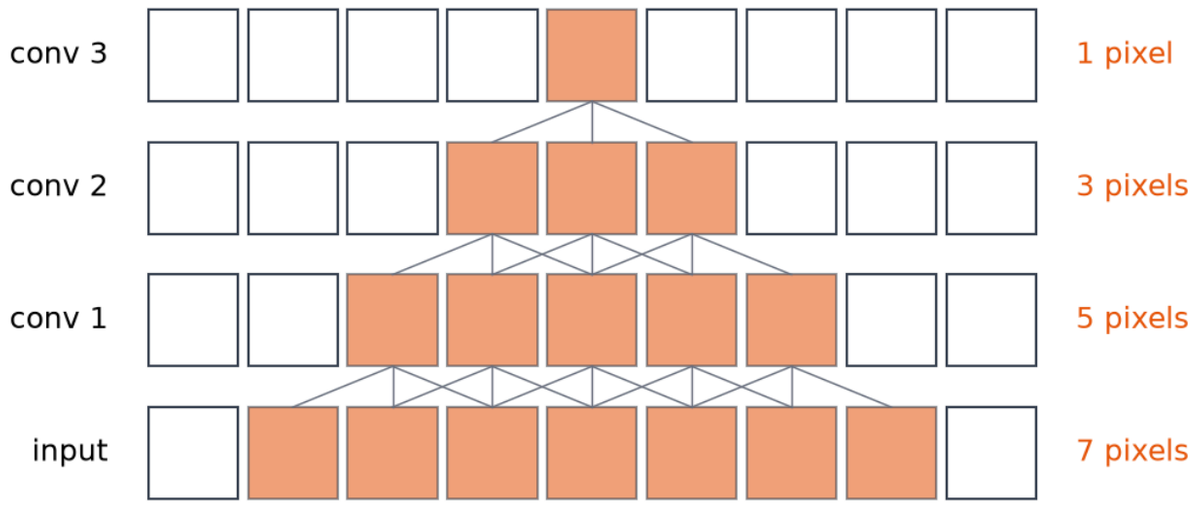
$$X = \begin{pmatrix} 1 & 3 & 2 & 0 \\ 4 & 6 & 1 & 1 \\ 0 & 2 & 5 & 7 \\ 1 & 1 & 3 & 2 \end{pmatrix}$$

Exercise 5 min pen and paper

1. Compute the 2 × 2 max pooling of X (stride 2).
2. Compute the 2 × 2 average pooling.
3. What is the output shape of `nn.MaxPool2d(2)` on a 64 x 32 × 32 tensor? How many parameters?

Your notes / answers (saved in this browser)

The receptive field grows with depth



One output of the third 3 × 3 convolution depends on 7 input pixels (per dimension).

The **receptive field** r of a neuron: the input region it depends on. Layer by layer, with j the distance (in input pixels) between neighbouring outputs:

$$r \leftarrow r + (k-1)\,j, \qquad j \leftarrow j \times s$$

Start with $r = 1$, $j = 1$. Stride and pooling multiply j: after them, every new layer enlarges r faster.

First layers see edges; deep layers see object parts, then objects.

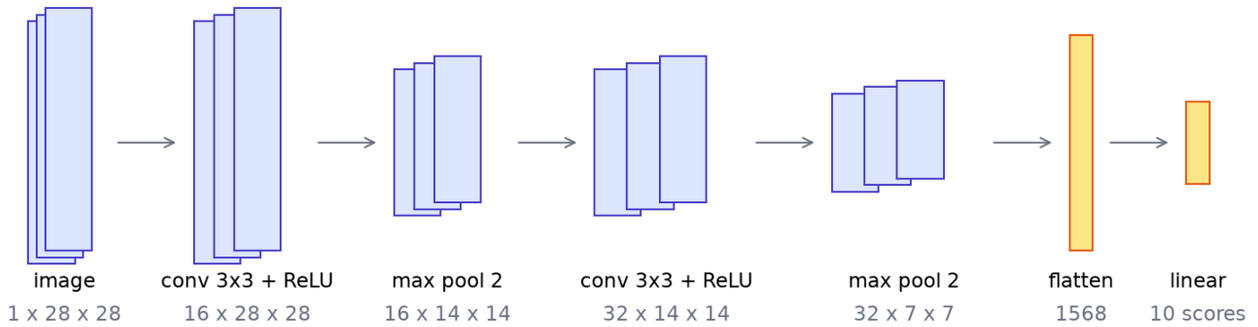
Exercise 5 · Receptive fields

Exercise 6 min pen and paper

1. Three stacked 3 × 3 convolutions (stride 1): what is the receptive field of the last one?
2. With C channels in and out at every layer, compare the number of weights of these three 3 × 3 layers with one 7 × 7 layer (ignore the biases). Which one has more non-linearities?
3. conv 3 × 3 (stride 1), then max pool 2 × 2 (stride 2), then conv 3 × 3 (stride 1): receptive field?

Your notes / answers (saved in this browser)

A convolutional network




```
cnn = nn.Sequential(
    nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),   # 16 x 14 x 14
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),  # 32 x 7 x 7
    nn.Flatten(), nn.Linear(32 * 7 * 7, 10))                      # 10 scores
print(sum(p.numel() for p in cnn.parameters()))                   # 20490
```

Inductive bias: what the architecture assumes before seeing data. **Locality** (a pixel is explained by its neighbours) and **weight sharing** (the same pattern can appear anywhere). Right assumptions = fewer parameters and better generalisation: 20 490 parameters versus 235 146 for the dense network of session 5.

From LeNet to VGG

Network	Year	Idea	Parameters
LeNet-5 (LeCun)	1998	conv, pool, conv, pool, dense: reads handwritten digits (cheques)	about 60 thousand
AlexNet (Krizhevsky, Sutskever, Hinton)	2012	the same recipe, bigger, ReLU, dropout, trained on GPUs: wins ImageNet (1.2 million images, 1000 classes) by a large margin	about 60 million
VGG-16 (Simonyan, Zisserman)	2014	only 3 × 3 convolutions, stacked in blocks, 16 layers (Exercise 5)	about 138 million
ResNet (He et al.)	2015	residual connections: 50, 101, 152 layers	ResNet-18: 11.7 million

The trend: deeper networks see larger receptive fields and build more abstract features. But going deeper is not free (after the TP).

Break: 10 minutes

TP · Convolutions by hand, then a CNN on FashionMNIST

TP 50 min tp/TP07_cnn.ipynb, Parts A to C

1. Part A: convolution, padding, stride and max pooling with NumPy loops, checked against `F.conv2d`; edge detectors on a real image (20 min).
2. Part B: parameters, shapes of the network of Exercise 3, receptive fields (10 min).
3. Part C: a small CNN class versus the dense network of session 5; then shift the test images by 3 pixels (20 min).

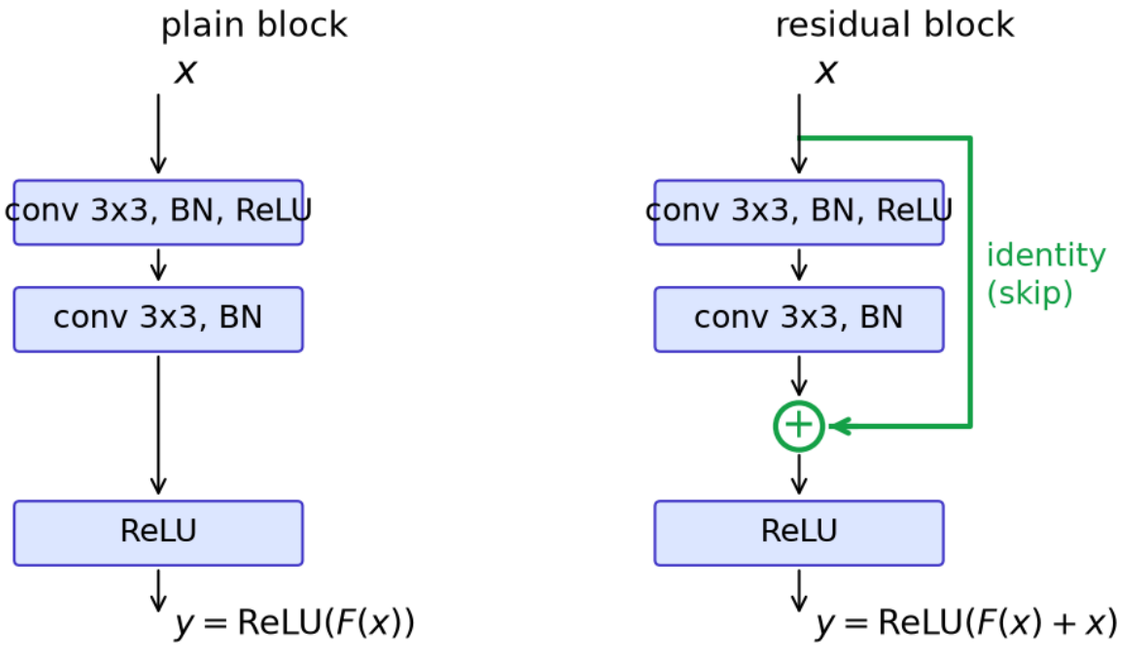
Deliverable: every cell up to Exercise 12 prints "Exercise validated!".

Your notes / answers (saved in this browser)

Deeper is not automatically better

- He et al. (2015), CIFAR-10: a plain 56-layer CNN has a **higher training error** than a 20-layer one. Not overfitting: an optimisation problem, the **degradation problem**.
- The deep network could copy the shallow one with identity extra layers; gradient descent does not find it.
- Idea: make the identity the default. Each block learns a **correction** $F(x)$ added to its input. A useless block only has to learn $F = 0$ (small weights).

$$y = F(x) + x$$



Why the gradient flows through a ResNet

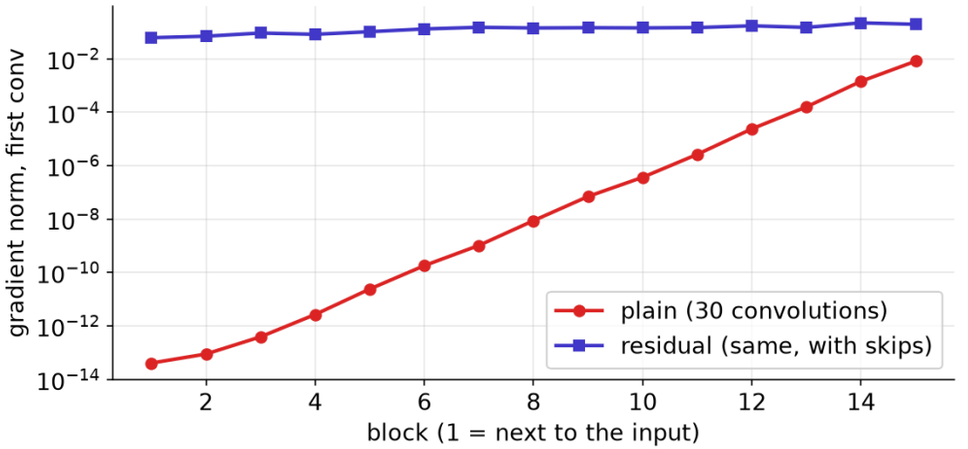
Derivative of one block $y = F(x) + x$:

$$\frac{\partial y}{\partial x} = \frac{\partial F}{\partial x} + I$$

Stacking blocks $x_{l+1} = x_l + F_l(x_l)$ from block l to the last block L:

$$x_L = x_l + \sum_{i=l}^{L-1} F_i(x_i) \;\Rightarrow\; \frac{\partial J}{\partial x_l} = \frac{\partial J}{\partial x_L}\left(I + \frac{\partial}{\partial x_l}\sum_{i=l}^{L-1} F_i\right)$$

The gradient reaching block l always contains $\partial J/\partial x_L$ itself, not multiplied by any weight or activation derivative: it cannot vanish through a product of small factors (session 5).

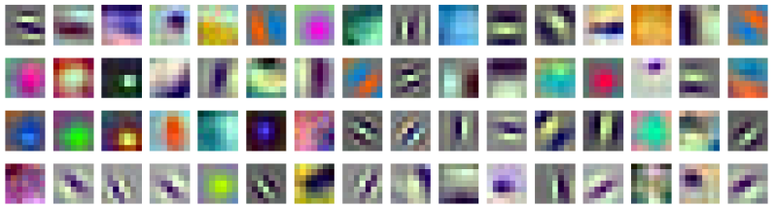


15 blocks at initialisation, no normalisation. Block 1: plain 4×10^{-14}, residual 6×10^{-2}.

Transfer learning: start from a pretrained network

- A ResNet trained on ImageNet has learned general features: edges and colours in the first layers, textures and parts in the middle, object-specific features at the end.
- For a new task with few images: keep the network, **replace the last layer** (1000 ImageNet classes) by a new one for our classes.
- Prepare the inputs like during pre-training: resize, then normalise each channel with the ImageNet mean $(0.485, 0.456, 0.406)$ and std $(0.229, 0.224, 0.225)$.

Your data	Strategy
Few images, similar to photos	freeze everything, train the new head
Few to medium, a bit different	then unfreeze the last block, small learning rate
Many images or very different (X-rays, paintings)	fine-tune everything, small learning rate



The 64 first-layer filters of ResNet-18: edges and colour blobs.

Transfer learning in PyTorch

```
from torchvision.models import resnet18, ResNet18_Weights

model = resnet18(weights=ResNet18_Weights.DEFAULT)    # 11 689 512 parameters, trained on ImageNet
for p in model.parameters():
    p.requires_grad = False                            # freeze the backbone
model.fc = nn.Linear(512, 10)                          # new head (trainable): 5 130 parameters
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
# ... train the head, then fine-tune the last block with a smaller learning rate:
for p in model.layer4.parameters():
    p.requires_grad = True
optimizer = torch.optim.Adam([{"params": model.layer4.parameters(), "lr": 1e-4},
                              {"params": model.fc.parameters(), "lr": 1e-3}])
```

- `requires_grad = False`: autograd computes no gradient for these weights, the optimiser does not move them.
- Parameter groups: one learning rate per group. The pretrained weights only need small adjustments.

TP · Transfer learning on CIFAR-10

TP 25 min tp/TP07_cnn.ipynb, Part D

1. Load ResNet-18 with its ImageNet weights, replace `fc`, freeze the backbone, count the trainable parameters.
2. Train the head on a CIFAR-10 subset (images upsampled to 64 × 64), then unfreeze `layer4` and fine-tune.
3. Going further: look at the first-layer filters and the feature maps.

Deliverable: the fine-tuned model beats the head alone; note both accuracies.

Hint: On a CPU each epoch takes 10 to 30 s; on Colab, choose a GPU runtime.

Your notes / answers (saved in this browser)

Watch it on YouTube Machine Learnia, Deep Learning series

Review the foundations of this course: the whole series.

Video 1 **Introduction**
Introduction to the Deep Learning series

Video 2 **Le perceptron et la régression logistique**
The perceptron and logistic regression

Video 3 **Les gradients calculés en détail**
The gradients, computed in detail

Video 4 **Un rappel sur la vectorisation**
A reminder on vectorisation

Video 5 **Développer le tout avec NumPy**
Coding it all in NumPy

Video 6 **Application : chiens et chats**
Application: cats versus dogs

Video 7 **Réseau de neurones à 2 couches**
A neural network with 2 layers

Video 8 **La backpropagation en détail**
Backpropagation in detail

Video 9 **Programmer le réseau à 2 couches avec NumPy**
Coding the 2-layer network in NumPy

Video 10 **Réseau de neurones à N couches**
A neural network with N layers

Playlist **The whole Deep Learning series**
10 videos, from the perceptron to the network with N layers

Videos in French. Click a line in your copy of the course to open the video.

Summary

1. A convolution slides a small learned filter over the image: locality and weight sharing, parameters independent of the image size.
2. Output size $\lfloor (n + 2p - k)/s \rfloor + 1$; a conv layer has $C_{\text{out}}(C_{\text{in}}k^2 + 1)$ parameters; pooling has none.
3. The receptive field grows layer after layer (faster after pooling and strides): stacks of 3 x 3 convolutions see far.
4. Residual connections $y = F(x) + x$ keep the gradient alive in very deep networks.
5. With little data, start from a pretrained ResNet: replace the head, freeze, then fine-tune.

Project: fine-tune a pretrained ResNet on your artworks and compare it with your dense baseline (tag `m3`).

Next session: data that come in sequences (text, time series). Recurrent networks, LSTM and attention.